

Documentation on merging cart and user measurements

Cart Collection Documentation

Overview

The Cart collection is used to store products selected by users before checkout. It acts as a temporary storage system where users can manage items they want to purchase.

The Cart system supports:

- Adding products to cart
- Updating product quantity
- Removing products from cart
- Clearing the entire cart
- Managing product variants such as size and color

Collection Name

`Cart`

File Name

`cart.js`

Documentation on merging cart and user measurements

Cart Schema Structure

```
module > JS cart.js > ...
1  const mongoose = require("mongoose");
2
3  const cartSchema = new mongoose.Schema({
4
5    user: {
6      type: mongoose.Schema.Types.ObjectId,
7      ref: "User",
8      required: true
9    },
10
11    items: [
12      {
13        product: {
14          type: mongoose.Schema.Types.ObjectId,
15          ref: "Product",
16          required: true
17        },
18
19        quantity: {
20          type: Number,
21          default: 1
22        },
23
24        size: {
25          type: String,
26          default: ""
27        },
28
29        color: {
30          type: String,
31          default: ""
32        }
33      }
34    ],
35
36    timestamps: true
37  });
38
39 module.exports = mongoose.model("Cart", cartSchema);
40
41
42
43
```

Field Description

Field	Type	Description
user	ObjectId	Reference to logged-in user
items	Array	Stores cart products
product	ObjectId	Reference to Product collection
quantity	Number	Number of products added
size	String	Selected product size
color	String	Selected product color
createdAt	Date	Cart creation time
updatedAt	Date	Last update time

Documentation on merging cart and user measurements

Cart Collection Relationships

Connected Collection	Relationship
User	Cart belongs to user
Product	Cart stores product references

Cart System API Documentation

Base Route

/api/cart

1. Add Product To Cart

Endpoint

POST /api/cart/add

Description

Adds a product to the user cart.

Request Body

```
{
  "productId": "PRODUCT_ID",
  "quantity": 1,
  "size": "M",
  "color": "Black"
}
```

Documentation on merging cart and user measurements

Success Response

```
{
  "success": true,
  "message": "Product added to cart"
}
```

2. Get User Cart

Endpoint

GET /api/cart/user/:userId

Description

Fetches all products added to the user cart.

Success Response

```
{
  "success": true,
  "cart": {}
}
```

3. Update Cart Product Quantity

Endpoint

PUT /api/cart/update/:productId

Description

Updates quantity of a specific product inside the cart.

Documentation on merging cart and user measurements

Request Body

```
{  
  "quantity": 3  
}
```

Success Response

```
{  
  "success": true,  
  "message": "Cart updated successfully"  
}
```

4. Remove Product From Cart

Endpoint

DELETE /api/cart/remove/:productId

Description

Removes a product from the cart.

Success Response

```
{  
  "success": true,  
  "message": "Product removed from cart"  
}
```

5. Clear Entire Cart

Endpoint

DELETE /api/cart/clear/:userId

Documentation on merging cart and user measurements

Description

Removes all products from the cart.

Success Response

```
{
  "success": true,
  "message": "Cart cleared successfully"
}
```

Cart System Security Considerations

- JWT authentication required
 - Only authenticated users can access cart
 - Product validation before insertion
 - Duplicate cart item prevention
 - Protected API routes
-

Cart System Scalability

The cart system is designed to support:

- High concurrent users
 - Fast product retrieval
 - Cloud database scaling
 - Future checkout integration
 - AI recommendation integration
-

Documentation on merging cart and user measurements

User Measurements Collection Documentation

Overview

The UserMeasurements collection stores body measurements of users to support AI-powered virtual try-on and personalized size recommendations.

The system helps improve:

- Virtual try-on accuracy
- Personalized fitting
- AI clothing recommendations
- Smart sizing systems

Collection Name

UserMeasurements

File Name

userMeasurements.js

Documentation on merging cart and user measurements

User Measurements Schema Structure

```
models > 25 userMeasurements.js > ...
1  const mongoose = require("mongoose");
2
3  const userMeasurementsSchema = new mongoose.Schema({
4
5    user: {
6      type: mongoose.Schema.Types.ObjectId,
7      ref: "User",
8      required: true,
9      unique: true
10   },
11
12   height: {
13     type: Number,
14     default: 0
15   },
16
17   weight: {
18     type: Number,
19     default: 0
20   },
21
22   chest: {
23     type: Number,
24     default: 0
25   },
26
27   waist: {
28     type: Number,
29     default: 0
30   },
31
32   hips: {
33     type: Number,
34     default: 0
35   },
36
37   shoulder: {
38     type: Number,
39     default: 0
40   },
41
42   inseam: {
43     type: Number,
44     default: 0
45   },
46
47   gender: {
48     type: String,
49     default: ""
50   },
51
52   measurementUnit: {
53     type: String,
54     default: "cm"
55   }
56 }, {
57   timestamps: true
58 });
59
60 module.exports = mongoose.model(
61   "UserMeasurements",
62   userMeasurementsSchema
63 );
```


Documentation on merging cart and user measurements

Field Description

Field	Type	Description
user	ObjectId	Reference to user
height	Number	User height
weight	Number	User weight
chest	Number	Chest measurement
waist	Number	Waist measurement
hips	Number	Hip measurement
shoulder	Number	Shoulder width
inseam	Number	Leg inseam measurement
gender	String	User gender
measurementUnit	String	cm/inches
createdAt	Date	Record creation time
updatedAt	Date	Last updated time

User Measurements Relationships

Connected Collection	Relationship
User	Measurements belong to user
TryOn	Measurements improve AI fitting
Image	Body scan image support

User Measurements API Documentation

Base Route

/api/measurements

1. Add User Measurements

Endpoint

POST /api/measurements/add

Documentation on merging cart and user measurements

Description

Stores user body measurements for AI virtual try-on support.

Request Body

```
{
  "height": 175,
  "weight": 70,
  "chest": 95,
  "waist": 82,
  "hips": 96,
  "shoulder": 45,
  "inseam": 78,
  "gender": "Male",
  "measurementUnit": "cm"
}
```

Success Response

```
{
  "success": true,
  "message": "Measurements saved successfully"
}
```

2. Get User Measurements

Endpoint

GET /api/measurements/:userId

Description

Fetches saved body measurements of the user.

Success Response

```
{
  "success": true,
  "measurements": {}
}
```

Documentation on merging cart and user measurements

3. Update User Measurements

Endpoint

PUT /api/measurements/update/:userId

Description

Updates existing user body measurements.

Request Body

```
{
  "waist": 84,
  "weight": 72
}
```

Success Response

```
{
  "success": true,
  "message": "Measurements updated successfully"
}
```

4. Delete User Measurements

Endpoint

DELETE /api/measurements/delete/:userId

Description

Deletes stored user body measurements.

Success Response

```
{
  "success": true,
  "message": "Measurements deleted successfully"
}
```

Documentation on merging cart and user measurements

}

Security Considerations

Cart System Security

- JWT authentication required
- Only authenticated users can access cart
- Prevent duplicate cart entries
- Validate product availability before adding

User Measurements Security

- Sensitive user data must be encrypted
- Access restricted to authenticated users
- HTTPS required for secure transmission
- Store minimum required personal data

Cloud Integration Recommendations

Cart System

- Redis caching for faster cart retrieval
- MongoDB Atlas for scalable cart storage
- Load balancing for high traffic

User Measurements

- Secure cloud database storage
 - AI integration for virtual try-on engine
 - Cloud backup for measurement data
-

Documentation on merging cart and user measurements

Scalability Considerations

The Cart and User Measurements systems should support:

- High concurrent users
- Real-time updates
- AI processing requests
- Fast API response times
- Cloud-native deployment

Recommended technologies:

- Node.js + Express
- MongoDB Atlas
- Redis
- AWS / Google Cloud
- Cloudinary for image handling